

Playing Games With My Editor for Fun and Profit

GEORGE ZAKHOUR

Abstract — Recent work has explored the relationship between programming and the well-being of programmers, from physical programming languages to systems that improve job security through planned obsolescence. Yet, an important contributor to mental health remains underexplored: play.

In this paper, we propose a technological approach to reconciling play and productivity by embedding gameplay within the appearance of legitimate work. We introduce a text-based game whose driver is a server implementing the Language Server Protocol thus integrating with any existing IDE.

CCS Concepts: • **Software and its engineering** → **Software creation and management**; *Programming languages*; • **Computing methodologies** → *Interactive games*; • **Computing education** → *Employment in computing*.

Additional Key Words and Phrases: Language Server Protocol, Text Editors, Employment, Games

1 INTRODUCTION

The last couple of SIGBOVIK proceedings have published papers concerned with the health of programmers. For example, Poololoop [24] is a programming language which is not written but instead physically walked, ran, cycled, or kayaked. The premise of that work is that an active programmer is an active worker and thus a productive one. In a second work [25], the author develops a planned obsolescence programming model which slows down programs based on their memory usage patterns. The consequence of that work is that the mental health of the programmer is improved vis-a-vis job security: when a team of programmers can continuously improve the program by discovering ever novel usage patterns or by slowing down competitors, then the bottom line is improved.

Nevertheless, multiple other factors besides job security contribute to mental health. None would argue that “just having fun” is not a great contributor to a good mental health. And what other ways is there to have fun than to play [2, 16]. Alas, play almost always brings no income which is yet another big contributor to mental health.

The answer might be obvious: to engage in recreational entertaining play while an income is being collected, simultaneously [15]. Alas, playing in the workplace is frowned upon and likely to deprive the player from an income [21].

While the obvious way to solve this problem is to draft a sociological manifesto about the need to play at work, build a collective who puts pressure on employers, have them unionise and spread to other employers, acquire local then national political offices, and finally draft the right to play in the labour laws of some country, this is a technological conference, and so the solution of interest is technological in nature.

While the legal pens are slowly being sharpened we propose to push the state-of-the-art in technology further for the purpose of gaming. This is not a novel or a radical proposition. For example, the first computer game, the text-based adventure *The Sumerian Game* was developed by Mabel Addis all the way back in 1962 [13]. She authored the game while she was a teacher as a way to engage her students in learning more about ancient civilizations.

The much lauded Sumerian Game doesn’t only serve as a success story for the use of technology for gaming, but also as an example where a hidden agenda was being executed in

the guise of gaming: learning. In this work we present a similar covert agenda: playing in the guise of working.

In this paper, we describe a game with state-of-the-art technology targeting a particular demographic: programmers who spend their time in fancy shmancy code editors. The game is designed so passerbys glancing at the screen of a programmer, expecting to see a code editor, would not be suspicious. Moreover, were the players asked about the nature of the activity they were engaging in by the passerbys, the programmer can truthfully reply with “working”.

Our solution is thus a text-based game about working, i.e. a work simulator, whose back-end is supported by a Language Server Protocol (LSP) server [17] which is portable and able to work across whichever IDE the programmer’s employer recommends.

Developing a game with which one interacts with a program in the front-end and a LSP in the back-end raises its own challenge. Therefore in this paper we outline these challenges and comment on how one would address them, as well as highlight the LSP features that make this endeavour worthwhile.

Contributions. In summary, our contributions are:

- The novel use of the Language Server Protocol [17] for non-programming recreational purposes, i.e. gaming, (Section 2)
- The OpenOffice game: a text-based role-playing solo adventure game where the solo player role plays as an office worker, precisely a programmer, who must balance getting real work done and fulfilling administrative tasks, (Section 3),
- `OpenOffice.py`, an implementation of the OpenOffice game in Python that implements the Language Server Protocol specification and usable out-of-the-box in any text editor or IDE, (Section 4), and,
- A NeoVim (Section 5.1) and VS Code (Section 5.2) plugins to play OpenOffice (Section 3) using the `OpenOffice.py` (Section 4) language server.

2 THE LANGUAGE SERVER PROTOCOL FOR GAMES

The Language Server Protocol has been proposed by Microsoft in 2016 [17]. It is based on top of JSON-RPC, and on top of HTTP¹. The protocol also loves to use long words where a shorter one would do [19], e.g. “character” instead of “char”, making reading² the requests, responses, and notifications easy³. Luckily though, the transport layer is surprisingly flexible and easy to use! You can use UNIX sockets, TCP sockets, or even standard input/output.

Nevertheless, nothing in the specification requires that the string being processed by an LSP server–client pair be a programming language. As long as the capabilities of the protocol are reinterpreted to fit a domain, then any LSP-capable client can be used as an interface with the server providing the logic.

The domain of interest in this paper is text-based role-playing solo adventure depressing games that force the user to navigate bureaucracy and administration in the hopes of getting work done for which they will be inevitably punished without really being able to win (Section 3).

¹If you were to strip HTTP of everything and only keep URLs, `\r\n` separators, and the single header `Content-Length`.

²By a human.

³With the exception of providing semantics for tokens for the purpose of syntax highlighting. Now, all of a sudden, you must send/process deltas in a large multiple-of-five bytestream.

To that end, in the following, we re-interpret the capabilities of the Language Server Protocol for the aforementioned domain.

Compared to traditional text-based games. Traditional text-based games are prompt-based, very much like modern-day GenAI-LLM-based AI-technology chatbots. The user writes a line and presses Enter. The line is sent to the server, is processed, and an answer is returned. Therefore, the natural progression in the timeline of the game is directly represented by the timeline of the request–response pairs: if the player wishes to do an action first and then a second, they send the first prompt and then the second.

In the LSP-way, this is not the case. A user is free to “go back in the prompts” and modify previous lines and change the current state they observed.

This requires rethinking the sorts of games that one can play with the help of LSPs. If a game requires the player to commit to a choice and suffer its consequences throughout the whole game, then this won’t do: the user may simply go back and delete or modify the offending action. We conjecture that the games suitable for LSP dynamics are games that cannot be won: ones where the player just have to last as long as they can. They are free to go back and change their decisions, but the combinatoric explosion of the cascading side-effects ought to prevent the player from doing so. Briefly, the game gives the player the ability to edit the history but the cost of doing so is too great.

Code Completions. Text-based games have always suffered from the freedom given to the player to type anything in the prompt [22]. That’s due to the prompt being an open-ended field which is processed at the end when a user submits the line as opposed to character-by-character.

The LSP-way is a paradigm shift. The player is no longer faced with a manual or a blank prompt. A pretty little dropdown now follows the player’s cursor telling him/her all the moves they could do, and which make sense. The player now has reasonable choices.

Hovering. After making a choice, whether it is to select an action or an object, the player may want to inspect the action for its side-effects or the object for its state. Hovering is the most suitable dynamic for this problem. Every game entity or action can be represented by an identifier and when the user hovers over `ghoul` or `pickup` they will see the stats of the `ghoul` and whether `pickup` up the `ghoul` would add too much weight to their pouch. If they hover over `hammer` they may even see the amount of damage it does, etc...

Goto Definition. Interacting with an identifier can go beyond just hovering over it. For example “going to basecamp” may be as simple as Ctrl+Click on the `basecamp` keyword. Or Ctrl+Click on `monster` may go to the round when the monster was introduced, from whence the whole interaction with it can be examined.

Find All References. While goto definition can be used to examine the history of interaction with an entity, a more suitable LSP capability can be used: find references. With find references, the LSP could return all of the locations where an interaction happened with a monster (directly or indirectly). The user then, instead of scanning a whole range of actions, can focus in a popup on only those that (in)directly affected the entity.

Publishing Diagnostics. The interaction with the LSP is not always one-way: the server may also publish messages to a client. This interaction is useful whenever spontaneous events happen. For example, the player may be taking too long to decide what to do in face of a rabid

bard. In which case the server may send a “diagnostic” to the player saying “The rabid bard couldn’t take it any longer and attacked you with his lute. You just received 10 damage.”.

It can also be used for the server to send “syntactic errors” back to the client. After all the player must type in the editor something and it is very likely that it’s neither syntactically correct nor complete.

But not all messages are errors. The server may send diagnostics at checkpoints summarizing the state of the player or the game. This way, the user may hover over the diagnostic message and keep track of the progression of the game.

3 OPENOFFICE: THE GAME

In this section we describe a solo role-playing adventure game designed to be played through a program with the help of a LSP server and an IDE.

The game is played in rounds in which the stats of the solo player are modified. In each round, the player receives tasks to complete and powerups that aid in completing them.

Player Stats. The player has five numerical statistics. All five numbers together indicate the *state* of the player. It is comprised of the following: (1) *health* (H), indicating how healthy the player is, (2) *expertise* (E), indicating the expertise level of the player, (3) *professionalism* (P), indicating the professionalism of the player, (4) *work points* (W), representing the amount of work the player is able to do, and, (5) *vacation points* (V), representing the amount of vacation the player may take.

Winning Criterion. The player shall never win. The grind never ends [23].

Losing Criteria. The player shall lose and the game shall terminate whenever *at least one* of the following conditions are true. (1) The player’s health is negative, this is referred to as *burning out*. (2) The player’s expertise does not increase after each round, this is referred to as *not driving innovation*. And, (3) the player’s accountability is negative, this is referred to as *not being a team player*.

Tasks. Tasks are split into two categories: administrative and work, and have cost points. Grinding on administrative tasks cost negative points to the player’s health as these are soulless brainless paper-shuffling box-ticking tasks. On the other hand, grinding on work tasks give positive points to the player’s expertise as these are challenging, fun, creative, and fulfilling.

The administrative tasks of OpenOffice are: renewing work visa (health penalty: 5, cost: D4+5 work points), emailing report (H:1, C:D2+1), attending a meeting (H:2, C:D2+3), replying to an email (H:1, C:D1+2), following up on an email (H:1, C:D1+1), filling a form from HR (H:2, C:D1+1), writing a LinkedIn post (H:5, C:D2+2), completing a mandatory training (H:3, C:D2+2), creating a checklist (H:1, C:D1+1), supervising a task that has been delegated (H:2, C:D2+2), and organizing a doodle on behalf of your manager (H:1, C:D2+1).

The work tasks of OpenOffice are: writing code (expertise bonus: 1, cost: D2+1), implementing a prototype (E:3, C:D3+1), fixing a bug (E:2, D2+1), and reviewing code (E:1, D4+1).

The only task which is both administrative and work is organizing a party (H:8, E:1, C:4+4).

Powerups. The game also grants the player random powerups to keep them engaged and playing longer. These are also split into two groups: instantaneous and selective.

The selective powerups shall be used once and apply to a task, at the player's choosing, to help in progressing it. Some of the tasks incur a health penalty (negative points) or a health bonus (positive). These powerups are: AI Wrote It (completes between D20-10 of a task), Team Supported (C:D3, H:-2), Assigned to Junior (C:D3, H:-1), Pet Therapy (C:D3-1,H:2), Stronger Coffee (C:2+D2,H:-3), Fruit Basket (C:D2,H:1), and finally Work from Home (C:D4-2, H:3).

The instantaneous powerups are three and two of them grant the player more vacation points: (1) Flooded Office (V+10), and Amazon AWS Outage (V+2). The third one is *HR on Holiday* which, if drawn, skips the *HR penalty* at the end of each round.

Before Each Round. The game is played in rounds. At the start of every round (W) is replenished to its full capacity. Crucially (V) remains identical to the one at the end of the previous round. Next, powerups are picked and those instantaneous ones are immediately acted upon. Next, tasks are picked.

The scheme by which the numbers of powerups and tasks are chosen is free for the player to decide. We recommend the following. For every expertise point, throw a D5 dice giving answers from 0 to 5 inclusive. Pick the largest two of the throws and add them together to get the number of tasks. To get the number of powerups, throw a D4 for every point of professionalism and pick the smallest two throws and add them.

During Each Round. During the round, the player may do any number of steps. A step is one of the following four. (1) Taking a vacation for some number of time, this number must be smaller than (V) and it is subtracted from it after completing the step. Additionally, as taking vacation is generally beneficial, the same number is added to (H) and subtracted from (W). (2) Grinding on a task for some number of time, this number must be smaller than the (W) and it is subtracted from it after completing the step. Whenever a work task is done then its expertise points are added to (E) and its health penalty is subtracted from (H). (3) Leveraging a powerup for a task, which applies the bonuses from a yet-unused powerup to the player and to the specified task. And finally, (4) delegating a task which, if successful, completes the task without acquiring its expertise points or health penalty. To decide if a task is successful, throw a D20 and add it to (P), if it is greater than 16 (inclusive) then it is successful. The player may not re-attempt to delegate any task. Regardless whether a delegation is successful or not, attempting a delegation decreases (P) by one.

After Each Round. After each round, the losing criteria is evaluated to determine whether more rounds shall be played or not. If all tasks were completed then (P) is increased by one. Moreover the *HR penalty* is computed if the HR is not on holiday, i.e. if the "HR went on holiday" powerup has not been drawn. First, (P) is decreased by the unused (W). And second, for each uncompleted task, (P) is reduced by n where n is the difference between the current round number and the round number when the task was assigned to the player.

4 OPENOFFICE.PY: THE IMPLEMENTATION

We have implemented OpenOffice (Section 3) in the Python programming language. In Section 4.1 we describe the front-end: the language in which the user plays. In Section 4.2 we describe the back-end: the game engine that implements the LSP protocol which parses and interprets the user actions. In Section 4.3 we describe which LSP methods the game engine supports and how they behave.

4.1 The Front-End

To design a deceptively good enough front-end language that can fool any passerby into thinking its user is working, we must understand how users work and are expected to work in the future. In the age of GenAI/LLM, programmers have been promoted from mere engineers to orchestrators and managers of fleets of agents [26]. The likelihood of the user churning out manually tens of lines of code per minute at 100 words-per-minute rate is close to zero [1]. The work that the modern and future programmer will do will mostly be locked inside their head and not observable to an outside observer: programming will be a purely mental exercise consisting of working out which systems to connect, which configurations to enable, and the interleavings of all these options.

In summary, programming will no longer be about *expression* but rather about *configuration*. This trend has not been kick-started by GenAI/LLM-assisted programming, it can be traced back to declarative programming where one expresses a target configuration that the system ought to reach, leaving out the technical engineering details to the system runtime [11].

For example, what used to be a system administrator became a DevOps [14] in the mid to early 10s. This group of administrator-programmers developed their own tooling which inevitably fell into the Turing tarpit [20]. Starting first with Ansible [7], the tooling quickly dissolved into Infrastructure-as-Code languages [8]. The quintessential example of infrastructure-as-code languages must be Terraform [5] which is programmed with its own HashiCorp Configuration Language (HCL) reminiscent of both the JSON [4] and YAML [3] configuration languages.

To that end, we design the front-end language of OpenOffice in a similar style. Below is an example.

```
1 round email impl ai # two tasks and one powerup have been drawn
2   - grind email 1
3   - vacation 2
4   - leverage ai for impl
5 end
6
7 # this is the second round
8 round meeting impl2 # two tasks have been drawn
9   - grind email 2
10  - grind meeting 3
11  - delegate impl
12 end
```

This code shows two rounds. In the first, two tasks are assigned and given names: email, for the task of sending an email, and impl, for the task of implementing a prototype. Moreover the AI assistance powerup has been drawn and assigned to ai. The second contains only two tasks: a meeting, and another implementation

The first round contains three steps: spending one work point on the email, taking 2 units of vacation, and using the AI assistance for the implementation⁴ which may not guarantee that the task is completely done. The second round contains three steps: grinding for two more work points on the email that the player was assigned in the previous round and a further three on the newly assigned meeting task. Finally, the player decides to delegate the first implementation

⁴Clearly the player is lazy in this round.

task which may or may not be successful. Meanwhile the second implementation task is left untouched⁵.

Formally-ish speaking, the grammar of the front-end language is this:

Definition 4.1 (The Syntax of the Game). A game expression g has the following shape:

$$\begin{aligned} g &::= \varepsilon \mid r\ g \\ r &::= \text{round } x_0 \dots x_n\ s\ \text{end} \\ s &::= \sigma \mid \sigma\ s \\ \sigma &::= -\text{grind } x\ n \mid -\text{vacation } n \mid -\text{leverage } x\ \text{for } n \mid -\text{delegate } x \end{aligned}$$

where x is the syntactic class of identifiers and n of numbers.

4.2 The Back-End

The biggest challenge for the back-end is nondeterminism. If a player decides to close the editor and resume later, how should the same state be recomputed and the same intermediary tasks and powerups be drawn? The back-end resolves this issue by seeding its pseudo-random number generator with the full path of the file being edited. Alas, this implies that were the game script to change location, the end state is unlikely to be the same.

The whole of `OpenOffice.py` back-end is a 888-line Python 3 program with a shebang as the first line. It depends on the standard packages that come in with Python: `typing`, `enum`, `dataclasses`, `itertools`, `random`, `math`, `sys`, `json`, `io`, and `getpass`⁶.

It is also annotated with types almost everywhere and `mypy` [10, 12] approves of it⁷. The types are there because the author cannot imagine how anyone can write more than 100 lines of Python and still be able to track what is what.

4.3 LSP Support

In this subsection we describe the LSP methods that the game engine supports.

4.3.1 Document Open and Modification. Obviously. The LSP server needs to know when a document is opened or modified.

One thing to note, we do not have any no real reason to be efficient, so whenever the user modifies the program we send the entire content again for the server to reprocess. That's right: on, every, single, key stroke. That's how efficient the Python interpreter is. It can keep up.

4.3.2 Semantic Tokens. I won't write a syntax highlighter for every single editor out there.

But I also care about my users. If a passerby sees you editing code that is not syntax highlighted that's a red flag. "What's that immature, silly, eyebrow-raising language that doesn't even have a syntax highlighter?" is what the passerby would be thinking if they saw you editing white-on-black, or worse black-on-white, text in an editor. "I better tell the CTO that they are not using industry-grade languages out here" is the next natural thought.

Therefore the LSP sends semantic token information to the client. In other words, each token in the source code gets a syntactic class which editors use different colors for. Right now, the

⁵Clearly this player is not going to last very long.

⁶Obviously to get the password of the user and send it across the wire to a mesh of zombie machines somewhere on the internet.

⁷Eventhough the author did observe runtime errors while running initial tests

keywords `grind`, `vacation`, `delegate`, `leverage`, and `for` are classified the same, as functions. The other two keywords `start` and `end` are classified the same, as keywords. All powerup identifiers are classified as enums. And all number literals are classified as strings.

4.3.3 Diagnostics. You need to know many things about the state of the game if you want to play it. For example, have you been fired yet or not? Or have you used that powerup or not? Or how do you even play this game?

The game engine pre-emptively sends diagnostics to the client answering these FAQs.

For example, the first character in the file always has the game spiel⁸ that the player can read to learn the objective of the game.

Moreover, on every syntactic or semantic error, an error diagnostic is sent to the client with the cause. For example, you attempted to grind on a completed task. Or you wanted to reuse a powerup. Or you wanted to leverage an email report for flooding the office⁹.

Additionally, warning diagnostics are sent. For example when a task is not yet completed or a powerup has not been used. These are nice because the client often renders these as yellow underlines. So you get to visually see what you can use.

Finally, info boxes are used in three cases. First, when leveraging a powerup a dice has to be rolled. The outcome of the dice throw is sent as an info. Similarly for delegating a task. And finally, at the end keyword, the state of the player is sent.

4.3.4 Hover. Hovering is useful to document the language. When the user hovers over a step action, `grind`, `vacation`, and so on, a popup will appear with a description of the command and how to use it. Hovering the `round` keyword lists all the tasks that were assigned to the player and the powerups that they drew. And finally hovering over tasks and powerups displays a description of the task or powerup and their stats, e.g. health hit points, or work points, etc.

4.3.5 Goto Definition. This LSP feature did not end up being so useful. Yet this dynamic was the genesis of this whole paper, so I wanted to keep it. Right now, go-to definition or Ctrl+Click on a task or powerup, sends you to the round in which the task or powerup was drawn.

4.3.6 Find References. This feature is ever so slightly more useful than go-to definition. Say I've been working in many steps on organizing a party and I want to see the history of these steps. It's easier to just find all references to the party and look at the list as a consistent whole.

4.3.7 Code Completion. This has got to be the coolest feature. Text-based adventure games have suffered through the liberty given to the user to type anything. Modern text-based games have fixed it by effectively giving the user buttons to click. But that's too restrictive and simple.

Code completion is my answer to this problem. When the user wants to start a new round, code completion already suggests the round `task0 task1 powerup0 . . . end` syntax which they can click. After typing `-` they will get a list of all four steps they can take with a description of what each does. After typing `grind` they will get a suggestion of all the open tasks left to be done with their work point cost besides.

Go try it, genuinely, it improves the UX tremendously.

⁸Wortspiel beabsichtigt.

⁹It's the other way around.

5 CASE STUDIES

For whatever reason, one cannot just tell an IDE or an editor: hey buddy, here's a binary that implement the LSP spec, use it while running this file¹⁰. Sure one may want to configure stuff, but shouldn't there be reasonable defaults? Anyways, I had to do quite a bit of work to get this thing running. So now you have to read through it. In exchange you get pretty pictures with colours in them¹¹.

5.1 NeoVim

I use Vim. Sadly vim does not talk LSP. Luckily, NeoVim did two things right: (1) a nice looking website [18], and (2) supporting LSP. Sadly, this must be done with Lua [9]. Luckily, only a few changes have to be made. First create a `lua/lsp_game.lua` file in your NeoVim config:

```
1 vim.api.nvim_create_autocmd({ "BufReadPost", "BufNewFile" }, {
2   pattern = "*.game",
3   callback = function()
4     vim.lsp.start({
5       name = "my-game-lsp",
6       cmd = { "PATH TO OpenOffice.py GOES HERE" },
7       root_dir = vim.fn.getcwd(),
8     })
9   end,
10 })
```

which will run `OpenOffice.py` on the existing buffer if its filename ends with the game extension. Then add this line `lua require('lsp_game')` in your `init.vim`.

As is visible in Figure 1, NeoVim does a pretty decent job. Eventhough it's a terminal application, and even though it has mouse support in the terminal¹², a hover event needs to be manually triggered with K. It also highlights in the margin the lines where diagnostic messages are attached.

5.2 VS Code

VS Code is just awful. Luckily, VS Code did one thing right: (1) it was the *raison d'être* for the Language Server Protocol [17]. Sadly, everything's written in Type/Java-Script. Sadly, you can't just write a file to make VS Code use your LSP server. Sadly, you need to make a whole VS Code plugin. Sadly, you need the internet to compile this plugin since you need to run some weird commands that download stuff. Sadly, you can't locally install your plugin. Sadly, everytime you need to run your plugin you will need to open VS Code to your plugin project, then press F5 to run some plugin development kit which opens a new VS Code window. Luckily, your LSP server is attached and working there.

¹⁰As the author believes is a reasonable thing to ask for.

¹¹Unless you are reading this from the Lulu printout which is likely in black-and-white. In which case open up your computer, you know that aluminum-cased book-like apparatus in your bag, and open <https://sigbovik.org/2026/proceedings.pdf>.

¹²It's 2026, of course terminal programs support mouse events. Vanilla Vim supports mouse since at least vim7 from all the way back in 2004! You don't trust me? That's when the `mouse` option was added to the options of vim: <https://github.com/vim/vim/blob/071d4279d6ab81b7187b48f3a0fc61e587b6db6c/runtime/doc/options.txt#L4017>. You still don't believe me? Open vim and run `:set mouse=a` and click and scroll to your heart's desire.

```

50 - grind supervise2 2
51 - grind training2 3
52 - grind training1 3
53 - grind meeting3 3
54 - vacation 13
55 - leverage coffee0 for party1
56 end
57
58 round supervise3 followup0 checklist3 proto2 proto3 meeti
end
for

```

(a) Autocomplete suggests how to start a new round. It shows the drawn tasks and powerups and suggests names for them.

```

57 - vacation 13
55 - leverage coffee0 for party1
56 end
57
58 round supervise3 followup0 checklist3 proto2 proto3 meeti
59 - grind
60 end

```

party1	Text	3
proto2	Text	3
proto3	Text	3
review3	Text	3
doodle1	Text	3
meeting5	Text	3
followup0	Text	3

(b) When expecting a task, autocomplete lists all the available ones with their cost. Here proto2 costs 3 work points.

```

51 - grind training2 3
52 - grind training1 3
53 - grind meeting3 3
54 - vacation 13
55 - leverage coffee0 for party1
56 end
57
58 round supervise3 followup0 checklist3 proto2 proto3 meeting5 review3 doodle1 aws0 pet0 juni
59 - grind supervise1 10
60 end

```

Diagnostics:

- Task supervise1 is already completed

(c) Diagnostics are helpful to display errors. Here supervise1 has already been completed and not possible to grind on. The diagnostic over end would display that the player has lost (cf Figure 2). Moreover, because party1 is not completed then it's underlined in yellow, and because ail is unused it is also underlined in yellow.

```

47 round code6 meeting3 checklist2 training1
48 - grind code6(Used) Stronger Coffee
49 - grind check
50 - grind superDouble espresso in one.
51 - grind train
52 - grind trainHealth Penalty: 3.
53 - grind meeti
54 - vacation 13Progress: between 2 and 4.
55 - leverage coffee0 for party1
56 end

```

(d) Hovering over a powerup displays if it was used or not, its name, its description, its health penalty, and the progress it could do on a task.

```

nd code6 meeting3 ch
rind code6 3
rind checklist2 1
rind supervise2 2
rind training2 3
rind training1 3
rind meeting3 3
vacation 13
verage coffee0 for party1

```

Organize a Party	
'Wouldn't it be fun to ge	
Work Points: 4	
Health Penalty: -8	
Expertise Bonus: 1	

(e) Hovering over a task displays its name, its description, its work point to complete, its health penalty, and the expertise bonus.

```

38 - grind supervise1 2
39 - grind doodle0 2
40 - grind email1 2
41 - grind email0 2
42 - grind review2 1
43 - grind code5 5
44 - vacation 5
main.game
1 main.game[34 col 42-52] round checklist1 doodle0 code5 training0 supervise1 review2 email0 email1 junior0
2 main.game[37 col 24-34] - leverage junior0 for supervise1
3 main.game[38 col 9-19] - grind supervise1 2

```

(f) Finding all the references to the task supervise1 lists three locations, one of them being the definition point.

Fig. 1. OpenOffice: The Game (Section 3), being played in NeoVim using OpenOffice.py (Section 4).

```

52 - grind training1 3
53 - grind meeting3 3
54 - vacation 13
55 - leverage coffee0 for party1
56 end
57
58 round supervise3 followup0 checklist3 proto2...

```

```

round supervise3 followup0 checklist3 proto2 proto3 meeting5 review3
- grind
end
  checklist3
  doodle1
  followup0
  meeting4
  meeting5
  party1
  proto2
  proto3
  review3
  supervise3
4

```

(a) Autocomplete suggests to start a new round. (b) The incomplete supervise3 costs 3 points.

```

49 - grind
50 - grind
51 - grind
52 - grind
53 - grind
54 - vacati
55 - levera
56 end
57
58 round su
59 - grind supervise1 10
60 end

```

Task supervise1 is already completed

(Completed) Supervise a Delegated Task

Delegating doesn't mean no work, it's just less of a liability.

Work Points: 0

Health Penalty: -2

Expertise Bonus: 0

View Problem (Alt+F8) No quick fixes available

ing5 review3 doodle1 aws0 pet0 junior1

main.game 2 of 32 problems

FIRED! You were not a team player!

(c) VS Code can display diagnostics not only just as NeoVim (cf Figure 1) with hover-activated popups and underlines, but also in inline popups.

```

47 round code6 meeting3 checklist2 training1
48 -
49 -
50 -
51 -
52 -
53 -
54 -
55 - leverage coffee0 for party1
56 end

```

Progress decreased by 2

Health decreased by 3

Use a powerup on a task.

leverage <powerup> for <task>

View Problem (Alt+F8) No quick fixes available

```

g3 checklist2 training1 partv1 training2 meeting
1
2
3
3
Organize a Party
'Wouldn't it be fun to get everyone together?' - Mana
Work Points: 4
Health Penalty: -8
Expertise Bonus: 1
for party1

```

(d) Hovering over leverage displays the actual dice roll that leads to the coffee0 powerup doing only 2 work point progress on party1. (e) Hovering over a task displays its name, its description, its work point to complete, its health penalty, and the expertise bonus.

```

37 - leverage junior0 for supervise1
38 - grind supervise1 2

```

main.game ~/Documents/sigbovik26/lsp/example - References (3)

31 - grind code4 7

32 end

33

34 round checklist1 doodle0 code5 training0 supervise1 review2 email0 email1 junior0

35 - grind checklist1 1

36 - leverage ai0 for training0

37 - leverage junior0 for supervise1

38 - grind supervise1 2

39 - grind doodle0 2

training0 supervise1 review2

junior0 for supervise1

- grind supervise1 2

(f) Finding all the references to the task supervise1 lists three locations, one of them being the definition point. The right-panel shows that a powerup has been used on it and two work points have been spent grinding on it.

Fig. 2. OpenOffice: The Game (Section 3), being played in VS Code using OpenOffice.py (Section 4).

I won't share all the code here because I frankly forgot the commands that summoned the files on my machine and there's plenty of them around my disk by now. But go look at the data-availability statement section at the end of the paper for the link to the repo where you will find the VS Code plugin source code. Nevertheless, [Figure 2](#) are some screenshots that should convince you that this is running.

6 RELATED WORK

The history of text-based games and their problems is summarized well in the *GET LAMP* documentary [\[22\]](#). Go watch that.

If you want to implement a LSP server, I would advise you to read the specification [\[17\]](#), or even better watch or read other people doing it. Someone even wrote a whole book about this [\[6\]](#) if you're into that.

DATA AVAILABILITY STATEMENT

The implementation of `OpenOffice.py` is licensed under the GNU GPL v3 public license and is available in the author's Gitlab repo: <https://gitlab.com/gzakhour/openoffice>. The code for the VS Code plugin and the NeoVim config are also there. I may or may not read, accept, acknowledge, or comment on issues, pull request, forks, emails, or extension packs.

REFERENCES

- [1] Domenico Amalfitano, Andreas Metzger, Marco Autili, Tommaso Fulcini, Tobias Hey, Jan Keim, Patrizio Pelliccione, Vincenzo Scotti, Anne Koziolk, Raffaella Mirandola, and Andreas Vogelsang. 2026. A Research Roadmap for Augmenting Software Engineering Processes and Software Products with Generative AI. *ACM Trans. Softw. Eng. Methodol.* (Jan. 2026). <https://doi.org/10.1145/3788879> Just Accepted.
- [2] Daniel Lucas Arenas, Anna Viduani, and Renata Brasil Araujo. 2022. Therapeutic use of role-playing game (RPG) in mental health: A scoping review. *Simulation & gaming* 53, 3 (2022), 285–311.
- [3] Oren Ben-Kiki, Clark Evans, and Ingy döt Net Ingerson. 2009. YAML Ain't Markup Language (YAML™) Version 1.2. <https://yaml.org/spec/1.2/spec.html>.
- [4] Tim Bray. 2017. The JavaScript Object Notation (JSON) Data Interchange Format. <https://www.rfc-editor.org/rfc/rfc8259>. RFC 8259.
- [5] Yevgeniy Brikman. 2022. *Terraform: up and running: writing infrastructure as code*. " O'Reilly Media, Inc."
- [6] Nadeeshaan Gunasinghe and Nipuna Marcus. 2021. *Language server protocol and implementation*. Springer.
- [7] Lorin Hochstein and Rene Moser. 2017. *Ansible: Up and Running: Automating configuration management and deployment the easy way*. " O'Reilly Media, Inc."
- [8] Michael Hüttermann. 2012. Infrastructure as code. In *DevOps for Developers*. Springer, 135–156.
- [9] Roberto Ierusalimsky. 2006. *Programming in lua*. Roberto Ierusalimsky.
- [10] T Jalaja and T Adilakshmi. 2022. CIA Tool Using Python and mypy for Python Applications. In *International Conference on Information and Management Engineering*. Springer, 615–626.
- [11] Michael Kifer and Yanhong Annie Liu. 2018. *Declarative logic programming: theory, systems, and applications*. Association for Computing Machinery and Morgan & Claypool.
- [12] Jukka Lehtosalo. 2019. Our Journey to Type Checking 4 Million Lines of Python. <https://dropbox.tech/application/our-journey-to-type-checking-4-million-lines-of-python>. Dropbox Engineering Blog.
- [13] J.M. Leonard and R.L. Wing. 1967. Advantages of Using a Computer in Some Kinds of Educational Games. *IEEE Transactions on Human Factors in Electronics* HFE-8, 2 (1967), 75–81. <https://doi.org/10.1109/THFE.1967.233315>
- [14] Mike Loukides. 2012. *What is DevOps?* " O'Reilly Media, Inc."
- [15] Heide Lukosch, Allison Littlejohn, and Anoush Margaryan. 2013. Simulation games for workplace learning. In *Technology-Enhanced Professional Learning*. Routledge, 158–167.
- [16] Alyssia Merrick, Wendy Wen Li, and Dan J Miller. 2024. A study on the efficacy of the tabletop roleplaying game Dungeons & Dragons for improving mental health and self-concepts in a community sample. *Games for*

- Health Journal* 13, 2 (2024), 128–133.
- [17] Microsoft. 2025. Language Server Protocol Specification, Version 3.17. <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/>. <https://microsoft.github.io/language-server-protocol/specifications/lsp/3.17/specification/> Accessed: 2026-03-18.
 - [18] Bram Moolenaar. 2017. 10 Questions with Vim’s Creator, Bram Moolenaar. Interview on Binpress. <https://www.binpress.com/vim-creator-bram-moolenaar-interview/> Accessed: 2026-03-18.
 - [19] George Orwell. 1946. Politics and the English Language. Essay.
 - [20] Alan J. Perlis. 1982. Special Feature: Epigrams on programming. *SIGPLAN Not.* 17, 9 (Sept. 1982), 7–13. <https://doi.org/10.1145/947955.1083808>
 - [21] Leonard Reinecke. 2009. Games at work: The recreational use of computer games during working hours. *CyberPsychology & Behavior* 12, 4 (2009), 461–465.
 - [22] Jason Scott. 2010. Get LAMP: The Text Adventure Documentary. Online video (YouTube). <https://www.youtube.com/watch?v=LRhbcDzbGSU> Documentary film available on YouTube.
 - [23] Adrianna N Wiley. 2023. “The grind never stops” mental health and expectations of productivity in the North American University. *Anthropologica* 65, 1 (2023), 1–24.
 - [24] George Zakhour. 2024. Getting Up and Running the λ -Calculus. In *Proceedings of the 2024 SIGBOVIK Conference*. Association for Computational Heresy, 20. <https://sigbovik.org/2024/proceedings.pdf>
 - [25] George Zakhour. 2025. Programmatic Planned Obsolescence. In *Proceedings of the 2025 SIGBOVIK Conference*. Association for Computational Heresy, 19. <https://sigbovik.org/2025/proceedings.pdf>
 - [26] Wenyu Zhao, Wenbin Shang, and Yang Liu. 2025. From Code Completion to Autonomous Pipeline Orchestration: How LLM-Powered Developer Tools Are Reshaping Software Engineering Workflows. *American Journal Of Big Data* 6, 05 (2025), 111–139.